



Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment[cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class[cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\{new\}} = g_{\{current\}} + 1$, $h_{\{new\}}$ using your heuristic, and $f_{\{new\}} = g_{\{new\}} + h_{\{new\}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.
5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

- **Uniform-Cost Search (UCS)** prioritizes nodes based *only* on the accumulated cost from the start node to the current node, denoted as $g(n)$. Because it has no knowledge of where the goal is, it expands uniformly in all directions, treating all paths with the same step cost equally.
- A *Search** prioritizes nodes based on the *total estimated cost* of the path, denoted as $f(n) = g(n) + h(n)$. It adds a heuristic function, $h(n)$, which estimates the remaining distance from the current node to the goal. By factoring in this "guess" of the future cost, A* is an *informed* search that actively prioritizes nodes that pull it physically closer to the goal, avoiding the blind circular expansion of UCS.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

If a heuristic is NOT admissible, it means the heuristic **overestimates** the path cost. If this happens, A* will mistakenly think a truly optimal path is too expensive to explore. It will skip over the shortest route, and the A* algorithm will lose its mathematical guarantee of optimality, likely returning a suboptimal, longer path instead.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

If the agent can move diagonally, moving one square up and one square right in a single diagonal step would only cost **1 step**. However, the Manhattan distance formula ($|x_1 - x_2| + |y_1 - y_2|$) would calculate this distance as $1 + 1 = 2$. Because the heuristic guesses the cost is 2, but the true cost is only 1, the heuristic is **overestimating** the true cost. Since an admissible heuristic must *never* overestimate the cost, Manhattan distance fails

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

A significantly stronger and highly effective heuristic for the multi-goal food problem is the **Minimum Spanning Tree (MST) Heuristic**.

To calculate this, the agent treats its own current position and all remaining food pellets as nodes on a graph. It then calculates the Minimum Spanning Tree—the shortest total length of lines needed to connect all of these nodes together without forming any closed loops (using Manhattan distance between the points).

Why it is strong and admissible: Because the agent must eventually travel to every single piece of remaining food to win, the true cost of completing the level will *always* be equal to or greater than the weight of the Minimum Spanning Tree. Therefore, the MST is guaranteed to be an **admissible** heuristic (it never overestimates the true cost), but it is a much "tighter" and more informed guess than just looking at the single closest food pellet, drastically reducing the number of nodes A* needs to explore.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation



FACULTY OF COMPUTING